DATABASE MANAGEMENT SYSTEMS (CT 652)

SUPPLEMENTARY ANSWER BANK

Topics found in PDF Question Bank NOT covered in previous answer banks

Chapter 7: Transactions & Concurrency Control | Chapter 8: Crash Recovery | Chapter 9: Advanced DB Concepts

CHAPTER 7 GAPS: Transactions & Concurrency Control

Q: Consider the following log contents when a crash occurs. Explain how recovery would be done for each state in log-based recovery. (a) <T0,start>,<T0,A,1000,950>,<T0,B,2000,2050> (b) same + <T0,commit>,<T1,start>,<T1,C,700,600> (c) same + <T1,commit>

Asked: 2080 Ashwin; 2076 Baisakh; 2074 Bhadra; 2070 Bhadra — appears almost every exam | Marks: [6]

Answer

Background: Immediate Database Modification Rules

In immediate database modification, changes are written to disk even before a transaction commits. On crash, the DBMS must: REDO committed transactions (re-apply new values forward) and UNDO uncommitted transactions (restore old values backward). The log format <Ti, Xj, V1, V2> means Ti wrote Xj, changing it from old value V1 to new value V2.

UNDO(Ti): scan log BACKWARD for Ti's records; for each <Ti,Xj,V1,V2> restore Xj = V1 (old value)
REDO(Ti): scan log FORWARD for Ti's records; for each <Ti,Xj,V1,V2> set Xj = V2 (new value)
Rule: if log has <Ti,commit> → REDO Ti; if no <Ti,commit> → UNDO Ti

The Three Log Scenarios

```
Common prefix (all three scenarios share this):
  <T0, start>
  <T0, A, 1000, 950>    T0 writes A: old=1000 new=950
  <T0, B, 2000, 2050>   T0 writes B: old=2000 new=2050

Scenario (a): CRASH here — only the 3 lines above
Scenario (b): + <T0, commit>  <T1, start>  <T1, C, 700, 600>  then CRASH
Scenario (c): + <T0, commit>  <T1, start>  <T1, C, 700, 600>  <T1, commit>  then
CRASH
```

Scenario (a): Crash after T0 wrote A and B, but BEFORE T0 commit

```
Log at crash:
  <T0, start>
  <T0, A, 1000, 950>
  <T0, B, 2000, 2050>
  *** CRASH ***
```

Analysis: T0 has <T0,start> but NO <T0,commit> → T0 is UNCOMMITTED → must UNDO T0.

```
Recovery Action — UNDO T0 (scan log BACKWARD):
  Step 1: Find <T0,B,2000,2050> → restore B = 2000 (old value V1)
  Step 2: Find <T0,A,1000,950>  → restore A = 1000 (old value V1)
  Step 3: Write <T0, abort> to log

Final state: A = 1000,  B = 2000  (original values restored)
T0's partial work is completely rolled back.
```

Scenario (b): Crash after T0 committed, T1 started and wrote C, but BEFORE T1 commit

```
Log at crash:
  <T0, start>
  <T0, A, 1000, 950>
  <T0, B, 2000, 2050>
  <T0, commit>            ← T0 committed
  <T1, start>
  <T1, C, 700, 600>
  *** CRASH ***
```

Analysis: T0 has <T0,commit> → REDO T0. T1 has <T1,start> but NO <T1,commit> → UNDO T1.

```
Recovery Action:
  REDO T0 (scan FORWARD):
    <T0,A,1000,950>  → set A = 950  (new value V2 — idempotent)
    <T0,B,2000,2050> → set B = 2050 (new value V2 — idempotent)

  UNDO T1 (scan BACKWARD):
    <T1,C,700,600>   → restore C = 700 (old value V1)
    Write <T1, abort> to log

Final state: A = 950,  B = 2050,  C = 700
T0's committed work is preserved; T1's partial work is rolled back.
```

Scenario (c): Crash after BOTH T0 and T1 committed

```
Log at crash:
  <T0, start>
  <T0, A, 1000, 950>
  <T0, B, 2000, 2050>
  <T0, commit>            ← T0 committed
  <T1, start>
  <T1, C, 700, 600>
  <T1, commit>            ← T1 committed
  *** CRASH ***
```

Analysis: Both T0 and T1 have commit records → REDO both. No UNDO needed.

```
Recovery Action — REDO T0 and T1 (scan FORWARD):
  <T0,A,1000,950>  → set A = 950
  <T0,B,2000,2050> → set B = 2050
```

```
  <T1,C,700,600>   → set C = 600

Final state: A = 950,  B = 2050,  C = 600
Both transactions' committed work is guaranteed on disk.
```

Scenario	Log at Crash	T0 Status	T1 Status	Recovery Action	Final State
(a)	T0 wrote A,B; no commit; crash	UNCOMMITTED	N/A	UNDO T0: B→2000, A→1000	A=1000 B=2000
(b)	T0 committed; T1 wrote C; no T1 commit; crash	COMMITTED	UNCOMMITTED	REDO T0: A→950 B→2050; UNDO T1: C→700	A=950 B=2050 C=700
(c)	T0 committed; T1 committed; crash	COMMITTED	COMMITTED	REDO T0: A→950 B→2050; REDO T1: C→600	A=950 B=2050 C=600

Q: Consider the following log contents when a crash occurs. Explain how recovery would be done using log-based recovery with checkpoints.
<T0,start>,<T0,B,2000,2050>,<T1,start>,<checkpoint{T0,T1}>,<T1,C,700,600>,<T1,commit>, <T2,start>,<T2,A,500,400>,<T0,B,2000>,<T0,abort>,<T2,A,500>,<T2,abort> CRASH
Asked: 2070 Bhadra Q9; 2073 Magh — also appears as variation in 2080 Ashwin | Marks: [6]

Answer

Log Contents

```
  <T0, start>
  <T0, B, 2000, 2050>          T0 writes B: old=2000 new=2050
  <T1, start>
  <checkpoint {T0, T1}>        Checkpoint: active transactions = {T0, T1}
  <T1, C, 700, 600>            T1 writes C: old=700 new=600
  <T1, commit>                 T1 commits
  <T2, start>
  <T2, A, 500, 400>            T2 writes A: old=500 new=400
  <T0, B, 2000>                T0 abort log (old value for B = 2000)
  <T0, abort>                  T0 aborts
  <T2, A, 500>                 T2 abort log (old value for A = 500)
  <T2, abort>                  T2 aborts
  *** CRASH ***
```

Step 1: Build Redo and Undo Lists

```
Start from checkpoint {T0, T1}:
  undo-list = {T0, T1}   (active at checkpoint)
  redo-list = {}

Scan log FORWARD from checkpoint:
  <T1, C, 700, 600>   → T1 in undo-list, no change to lists
  <T1, commit>         → move T1 from undo-list to redo-list
                          undo-list = {T0},  redo-list = {T1}
```

```
  <T2, start>          → add T2 to undo-list
                          undo-list = {T0, T2},  redo-list = {T1}
  <T2, A, 500, 400>   → T2 in undo-list, no change
  <T0, abort>          → remove T0 from undo-list (already aborted)
                          undo-list = {T2},  redo-list = {T1}
  <T2, abort>          → remove T2 from undo-list (already aborted)
                          undo-list = {},  redo-list = {T1}

Final: redo-list = {T1},  undo-list = {} (both T0 and T2 already aborted in
log)
```

Step 2: Redo Phase (T1 only)

```
Scan log FORWARD from checkpoint, apply writes for T1:
  <T1, C, 700, 600>  → T1 in redo-list → set C = 600

T0's B write before checkpoint was already undone in log (T0 aborted)
T2's A write was already undone in log (T2 aborted)
```

Step 3: Undo Phase

Since undo-list is empty (both T0 and T2 already have abort records in the log), no additional undo is needed. Their abort records serve as proof they were already rolled back.

```
If abort records were NOT present, we would undo:
  UNDO T0: restore B = 2000 (scan backward, find <T0,B,2000,2050>, use
V1=2000)
  UNDO T2: restore A = 500  (scan backward, find <T2,A,500,400>, use V1=500)
  Write <T0,abort> and <T2,abort> to log
```

Key insight: Since T0 and T2 abort records ALREADY exist in the log, their undo is already done. The recovery algorithm recognises abort records and removes those Tis from the undo-list. Final database state: B = 2000 (T0 undone), C = 600 (T1 committed), A = 500 (T2 undone)

Q: Consider the schedule S: r1(y); r3(z); w1(y); w2(z); r3(y); w2(y). Determine whether S is conflict serializable or not. If serializable, find the equivalent serial schedule using a precedence graph.

Asked: 2079 Chaitra Q7a; similar problems in 2072 Magh, 2071 Bhadra | Marks: [4]

Answer

Step 1: List All Operations in Order

```
Step  Operation  Transaction
  1   r1(y)      T1 reads y
  2   r3(z)      T3 reads z
  3   w1(y)      T1 writes y
  4   w2(z)      T2 writes z
```

```
  5   r3(y)       T3 reads y
  6   w2(y)       T2 writes y
```

Step 2: Find All Conflicting Pairs

Two operations conflict if they (a) belong to different transactions, (b) access the same data item, and (c) at least one is a WRITE.

```
Data item y:
  w1(y) → r3(y): T1 writes y BEFORE T3 reads y  → edge T1 → T3
  w1(y) → w2(y): T1 writes y BEFORE T2 writes y  → edge T1 → T2
  r3(y) → w2(y): T3 reads y BEFORE T2 writes y   → edge T3 → T2

Data item z:
  r3(z) → w2(z): T3 reads z BEFORE T2 writes z   → edge T3 → T2

Conflict edges: T1→T3,  T1→T2,  T3→T2
```

Step 3: Draw Precedence Graph

```
                    T1
                  /  \
                v     v
              T3 -> T2

        Edges: T1 → T3,  T1 → T2,  T3 → T2
```

Step 4: Check for Cycle

Inspect the precedence graph for cycles. Starting from any node: T1 → T3 → T2. There is NO path from T2 back to T1 or T3. The graph is a Directed Acyclic Graph (DAG).

```
No cycle exists in the precedence graph.
Therefore: Schedule S IS conflict serializable.

Equivalent serial schedule (topological sort of DAG):
  T1 → T3 → T2
  i.e., execute T1 completely, then T3 completely, then T2 completely.
```

Q: A precedence graph has nodes T1, T2, T3, T4, T5 with edges: T1→T2, T1→T4, T2→T3, T3→T4, T4→T1, T2→T5, T3→T5, T4→T5. Is the corresponding schedule conflict serializable? Explain.

Asked: 2079 Jestha; also 2077 Chaitra — precedence graph with 5 transactions | Marks: [4]

Answer

Given Precedence Graph

```
Edges: T1→T2, T1→T4, T2→T3, T3→T4, T4→T1, T2→T5, T3→T5, T4→T5

                  T1 ————————————————————> T4
                 |                          |   \
                v                          v    v
               T2 ————————————> T3 ———————>T4   T5
                     \          \                ^
                      v          v               |
                    T5         T5  (T4→T1 creates cycle!)

          Key: T1→T4→T1 via path T1→T4 and T4→T1 forms a CYCLE
```

Cycle Detection

```
Trace for cycle starting at T1:
  T1 → T4  (direct edge T1→T4)
  T4 → T1  (direct edge T4→T1)
  Back to T1! → CYCLE FOUND: T1 → T4 → T1

Another cycle: T1 → T2 → T3 → T4 → T1
  T1→T2→T3→T4→T1 is also a cycle (length 4)
```

Conclusion

Since the precedence graph contains a cycle (T1 → T4 → T1), the schedule is NOT conflict serializable. A conflict serializable schedule requires the precedence graph to be a DAG (Directed Acyclic Graph — no cycles). The cycle indicates contradictory ordering requirements: T1 must come before T4 AND T4 must come before T1, which is impossible in any serial schedule.

Q: Explain how the graph-based (tree) protocol maintains concurrent execution of transactions. How is it different from two-phase locking?

Asked: 2079 Ashwin Q7b; 2077 Chaitra Q7b | Marks: [4]

Answer

Motivation for Graph-Based Protocols

Two-phase locking (2PL) guarantees conflict serializability but requires transactions to hold locks until the end, reducing concurrency. Graph-based protocols exploit prior knowledge about the order in which data items will be accessed to allow earlier lock release.

Prerequisite: Database Graph

A database graph (or data graph) is imposed on all data items. It is a directed acyclic graph where an edge from data item A to data item B means that any transaction that accesses B must have previously accessed A. The tree protocol is the simplest special case where this graph is a rooted tree.

Tree Protocol Rules

```
Tree Protocol — 4 Rules:

Rule 1: The first lock by a transaction Ti may be on any data item.

Rule 2: Subsequently, Ti can lock a data item Q ONLY IF
        Ti currently holds a lock on Q's PARENT in the tree.

Rule 3: Data items may be UNLOCKED at any time
        (unlike 2PL which must wait until shrinking phase).

Rule 4: A data item that has been locked and then UNLOCKED by Ti
        CANNOT be re-locked by Ti.
```

Example

```
                              Database Tree:        A
                                                  /   \
                                                B     C
                                              / \     \
                                            D   E     F

                        Transaction T1 wants to access D, E:
                       Lock A  (first lock — any item allowed)
       Lock B  (parent of D is B, and T1 holds lock on A which is parent of B)
                         Lock D  (parent is B, T1 holds B) ✓
                            Unlock A (can release early!)
                         Lock E  (parent is B, T1 holds B) ✓
                             Unlock B, Unlock D, Unlock E
```

Tree Protocol vs 2PL

Aspect	Two-Phase Locking (2PL)	Tree Protocol
Lock order	Any order	Must follow parent-before-child tree order
Unlock timing	Not until shrinking phase begins	Can unlock at ANY time after locking
Re-locking	Allowed before shrinking phase	NEVER allowed once unlocked
Deadlock	Possible — requires prevention/detection	IMPOSSIBLE — prevents deadlock inherently
Starvation	Possible	Possible
Guarantees	Conflict serializability	Conflict serializability + deadlock freedom
Concurrency	Lower (hold locks long)	Higher (early unlock allowed)
Prerequisite	None	Must know data access order in advance

Key advantage of Tree Protocol: deadlock is impossible because locks are always acquired top-down in the tree, so circular wait cannot occur.
Key limitation: a transaction may need to lock data items it does not actually use (to follow tree path from root to target item).

CHAPTER 8 GAPS: Crash Recovery

Q: What is a remote backup system? Explain its architecture, purpose, and how it ensures availability. What are hot-spare and warm-spare configurations?
Asked: 2079 Chaitra Q8; 2077 Chaitra Q8a; 2075 Baisakh Q8b; 2073 Bhadra Q6b; 2070 Bhadra Q7b — appears in almost every exam | Marks: [3-4]

Answer

Remote Backup System

A remote backup system (also called a remote site recovery system) maintains a geographically separate copy of the database at a backup site, which mirrors the primary site in real time (or near real time). If the primary site is destroyed by a disaster (fire, flood, earthquake, power grid failure), the backup site can take over and continue operations with minimal data loss and downtime.

Architecture

```
              PRIMARY SITE                          BACKUP SITE
┌──────────────────────────┐            ┌──────────────────────────┐
│  Active DBMS             │            │  Standby DBMS            │
│  Active transactions     │            │  Mirror of primary DB    │
│  Log buffer              │──Log──────▶ Receives & applies logs  │
│  Stable log storage      │  shipping  │  Stable log storage      │
│  Database files          │            │  Database files (copy)   │
└──────────────────────────┘            └──────────────────────────┘
                  │                                    │
                  └──────────────── Network ───────────────────┘
                                (WAN / dedicated link)

        On primary site failure → backup site activates and takes over
```

How It Works — Log Shipping

- The primary site continuously ships log records to the backup site as transactions execute
- The backup site receives log records and applies them to its local database copy (redo operation)
- The backup site's database is always a recent version of the primary database
- On primary failure, the backup site completes recovery (applying any remaining logs) and opens for service

Hot Spare vs Warm Spare

Configuration	Description	Log Application	Switchover Time	Cost
Hot Spare	Backup site applies logs in real time; database always current and open	Continuous, real-time	Seconds to minutes	Highest

Warm Spare	Backup site receives logs but applies them periodically (e.g., hourly)	Batch / periodic	Minutes to hours	Medium
Cold Spare	Backup site has tapes/dumps but no live mirror	Manual restore needed	Hours to days	Lowest

Recovery Time and Data Loss Objectives

```
RTO (Recovery Time Objective): How quickly the system must be back online
RPO (Recovery Point Objective): How much data loss is acceptable

Hot spare:  RTO = minutes,  RPO = near zero (seconds of data loss)
Warm spare: RTO = hours,    RPO = minutes to hours of data loss
Cold spare: RTO = days,     RPO = hours to days of data loss
```

Ensuring Consistency at Backup Site

- When primary crashes, backup site may have an incomplete log — it runs normal log-based recovery (redo committed, undo active transactions)
- Once consistent state is reached, backup site opens for read/write access
- Lost transactions: those whose log records were in transit when crash occurred and never reached backup — these are permanently lost (bounded by RPO)

Remote backup systems protect against site disasters but NOT against logical errors (accidental DELETE). For logical errors, point-in-time recovery from archived logs is used.
Real-world implementations: Oracle Data Guard (hot standby), PostgreSQL Streaming Replication, MySQL Replication.

Q: Suppose following log contents are present when a crash occurs. Explain what happens for a log-based recovery. Initial values: A=1000, B=2000, C=700.
<T0,start>,<T0,B,2000,2050>,<T1,start>,<checkpoint{T0,T1}>,<T1,C,700,600>,<T1,commit>,<T2,start>,<T2,A,500,400>,<T0,B,2000>,<T0,abort>,<T2,A,500>,<T2,abort> — then CRASH
Asked: 2070 Bhadra Q9 (6 marks); variation in 2073 Magh | Marks: [6]

Answer

Full Log Trace

```
1.  <T0, start>
2.  <T0, B, 2000, 2050>       T0: B changed from 2000 → 2050
3.  <T1, start>
4.  <checkpoint {T0, T1}>     Checkpoint taken; active={T0,T1}; dirty pages
flushed
5.  <T1, C, 700, 600>         T1: C changed from 700 → 600
6.  <T1, commit>              T1 committed
7.  <T2, start>
8.  <T2, A, 500, 400>         T2: A changed from 500 → 400
9.  <T0, B, 2000>             T0 abort log: undo record for B (CLR — B restored to
2000)
10. <T0, abort>               T0 aborted
```

```
11. <T2, A, 500>              T2 abort log: undo record for A (CLR — A restored to
500)
12. <T2, abort>               T2 aborted
    *** CRASH ***
```

Recovery Algorithm — Step by Step

Step 1: Locate Most Recent Checkpoint

Find <checkpoint {T0, T1}> at line 4. Active list L = {T0, T1}.

```
Initialize: undo-list = {T0, T1},  redo-list = {}
```

Step 2: Scan Forward from Checkpoint to Crash

```
Line 5:  <T1,C,700,600>    T1 in undo-list → no change
Line 6:  <T1,commit>       Move T1: undo→redo  undo={T0}, redo={T1}
Line 7:  <T2,start>        Add T2 to undo     undo={T0,T2}, redo={T1}
Line 8:  <T2,A,500,400>    T2 in undo-list → no change
Line 9:  <T0,B,2000>       CLR (compensation) — T0 already being aborted
Line 10: <T0,abort>        Remove T0 from undo  undo={T2}, redo={T1}
Line 11: <T2,A,500>        CLR — T2 already being aborted
Line 12: <T2,abort>        Remove T2 from undo  undo={}, redo={T1}

FINAL LISTS: undo-list = {}  redo-list = {T1}
```

Step 3: Redo Phase (scan FORWARD from checkpoint)

```
Only T1 is in redo-list:
  Line 5: <T1,C,700,600> → T1 in redo-list → set C = 600

T0 and T2 are NOT in redo-list; their writes are NOT redone.
```

Step 4: Undo Phase

```
undo-list is EMPTY — both T0 and T2 already have abort records in the log.
Their data items were already restored in the log (CLR records lines 9,11).
No further undo needed.
```

Final Database State After Recovery

Item	Before Any Transactions	After Recovery	Explanation
A	500	500	T2's write (500→400) was undone by T2's abort record
B	2000	2000	T0's write (2000→2050) was undone by T0's abort record
C	700	600	T1's write (700→600) was committed — retained by REDO

CHAPTER 9 GAPS: Advanced Database Concepts

Q: Explain the types of distributed databases: homogeneous and heterogeneous. What is a federated database system?
Asked: 2079 Jestha Q10b; 2075 Baisakh Q10b; 2073 Magh Q9b; 2072 Magh Q10a; 2071 Magh Q10b | Marks: [4]

Answer

Types of Distributed Databases

Distributed databases are classified based on the degree of uniformity across the participating sites. The two main types are homogeneous and heterogeneous.

1. Homogeneous Distributed Database

In a homogeneous distributed database, all sites use the same DBMS software (same vendor, same version or compatible versions), the same data model, and the same schema style. The sites cooperate fully, and the distribution is largely transparent to users.

```
Characteristics:
  • All sites run identical or compatible DBMS software
    (e.g., all sites run Oracle 19c, or all run PostgreSQL 15)
  • Same data model at all sites (all relational, all use SQL)
  • Same or compatible query language
  • Global schema is easy to maintain
  • Transparency: users query as if one unified database

Example: A bank with 50 branches, all running the same Oracle DBMS,
  with each branch storing its own customer data.
  A query like SELECT * FROM customers can retrieve from all sites.

Advantages: easier administration, simpler query optimization, full
transparency
Disadvantages: forces all sites to use same software; costly migration
```

2. Heterogeneous Distributed Database

In a heterogeneous distributed database, different sites use different DBMS software, different data models, or different hardware. Integration is much more complex because each site may use different query languages, data types, and schemas.

```
Characteristics:
  • Different sites run different DBMS (Oracle at site A, MySQL at site B,
    PostgreSQL at site C, MongoDB at site D)
  • May use different data models (relational at one site, NoSQL at another)
  • Different query languages, data formats, naming conventions
  • Requires middleware / translation layer to integrate
  • Partial or no transparency — users may be aware of differences

Example: A hospital group where:
```

```
  • Main hospital: Oracle relational DB for patient records
  • Pharmacy: MySQL for drug inventory
  • Lab system: MongoDB for test results (JSON format)
  • Legacy ward system: old hierarchical IMS database

Advantages: sites can choose best-fit DBMS independently
Disadvantages: very complex integration, limited transparency, performance
overhead
```

3. Federated Database System

A federated database system is a special type of heterogeneous distributed database where each component database (called a component DB or local DB) retains its full autonomy and is only partially integrated. Sites cooperate voluntarily and share only part of their data.

- Tight integration: full global schema; all sites must agree on unified schema
- Loose integration: each site has its own schema; queries are routed to individual sites; partial transparency only
- No integration: databases are completely independent; manual data exchange

Feature	Homogeneous	Heterogeneous	Federated
DBMS software	Same at all sites	Different at each site	Different, retains autonomy
Data model	Same (usually relational)	May differ	May differ
Schema	Global unified schema	Complex mapping needed	Each site has own schema
Transparency	Full transparency possible	Limited transparency	Partial transparency
Complexity	Low	High	Very high
Autonomy	Low (central control)	Medium	High (sites are independent)
Example	Oracle RAC cluster	Bank + hospital integration	University consortium DB

Q: What is a spatial database? Explain its features, data types, and applications. What is spatial indexing?

Asked: 2075 Bhadra Q9b; 2075 Baisakh; 2073 Magh Q9b; 2073 Bhadra Q9b; 2072 Magh; 2072 Ashwin Q10; 2071 Bhadra; 2070 Bhadra — appears as short note in most exams | Marks: [2-4]

Answer

Spatial Database

A spatial database is a database that stores, manages, and queries data that has a geometric or geographic dimension — data that describes the location, shape, and extent of objects in space (2D or 3D). Spatial databases extend the relational model with special data types and operations to handle geometric objects like points, lines, and polygons efficiently.

Spatial Data Types

Data Type	Description	Example
Point	A single location in 2D/3D space (x,y) or (x,y,z)	GPS coordinates: (27.7172, 85.3240) = Kathmandu
Line / LineString	A sequence of connected points forming a path	A road, river, or cable route
Polygon	A closed region bounded by line segments	A city boundary, lake outline, building footprint
MultiPoint	A collection of Point objects	Multiple ATM locations
MultiPolygon	A collection of Polygon objects	Country made of islands (e.g., Indonesia)
GeometryCollection	Mixed collection of any spatial types	A map layer with roads + buildings + parks

Spatial Operations

- Distance(P1, P2): compute distance between two spatial objects
- Intersects(A, B): does geometry A intersect geometry B?
- Contains(A, B): does polygon A completely contain B?
- Within(A, B): is A completely within B? (inverse of Contains)
- Area(P): compute area of a polygon
- Buffer(G, d): create a zone of width d around geometry G
- Union(A, B): merge two geometries into one

Spatial Indexing — R-Tree

Regular B+ tree indexes cannot handle multi-dimensional queries efficiently. Spatial databases use specialized indexes:

```
R-Tree (Region Tree): the standard spatial index
  • Groups nearby spatial objects into Minimum Bounding Rectangles (MBRs)
  • MBRs at each level contain all objects in their subtree
  • Similar to B+ tree but for 2D rectangles
  • Query: 'find all restaurants within 2km of my location'
    → Check only R-tree nodes whose MBR overlaps the query region

Quad-Tree: divides 2D space into four quadrants recursively
  • Good for uniformly distributed point data
  • Depth varies based on point density
```

Applications of Spatial Databases

- Geographic Information Systems (GIS): maps, land records, urban planning
- Navigation systems: GPS routing, shortest path (Google Maps, OpenStreetMap)
- Location-based services: 'find nearest ATM', 'restaurants within 500m'
- Environmental monitoring: tracking forest cover, flood zones, pollution
- Epidemiology: disease spread mapping, hospital proximity analysis

- Autonomous vehicles: real-time spatial processing for navigation
- Real estate: property boundary management, zoning analysis

Spatial SQL Example (PostGIS)

```
-- Find all hospitals within 5km of a given point:
SELECT name, address
FROM hospitals
WHERE ST_Distance(
  location,
  ST_GeomFromText('POINT(85.3240 27.7172)', 4326)  -- Kathmandu coords
) < 5000;   -- 5000 meters

-- Find all roads that intersect a flood zone polygon:
SELECT r.road_name
FROM roads r, flood_zones f
WHERE ST_Intersects(r.geometry, f.geometry)
  AND f.zone_name = 'Bagmati River Zone';
```

PostGIS (PostgreSQL extension) and Oracle Spatial are the most widely used spatial database systems.
The OGC (Open Geospatial Consortium) provides the standard for spatial data types and SQL extensions (ISO 19125).

Q: What is a parallel database architecture? Explain the shared-memory, shared-disk, and shared-nothing architectures. What are the advantages and limitations of each?
Asked: 2079 Chaitra Q9b; 2077 Chaitra Q9b; 2072 Magh Q10b; 2071 Magh Q10b; 2070 Bhadra Q10 — appears as short note | Marks: [3]

Answer

Parallel Database Architecture

A parallel database system uses multiple processors and disks working simultaneously to execute a single query or transaction faster. Parallelism improves performance for large-scale data processing. The three main architectures differ in how processors share memory and disk:

```
                    THREE PARALLEL DATABASE ARCHITECTURES

      SHARED-MEMORY          SHARED-DISK           SHARED-NOTHING
   ┌─────────────────┐    ┌─────────────────┐    ┌─────────────────┐
   │ P1  P2  P3  P4  │    │ P1  P2  P3  P4  │    │ P1 │ P2 │ P3 │
   │  \  |  /   /    │    │  |   |   |   |  │    │ M1 │ M2 │ M3 │
   │   Shared RAM    │    │  Interconnect   │    │ D1 │ D2 │ D3 │
   │  D1 D2 D3 D4    │    │  Shared Disks   │    └────┴────┴────┘
   └─────────────────┘    └─────────────────┘    Each node has its own
 All processors share   All procs share disk   private memory & disk
            same memory & disk     but have own memory
```

1. Shared-Memory Architecture

- All processors share a single global main memory (RAM) and all disk storage
- Communication between processors is via the shared memory — very fast
- OS-level hardware support needed (SMP — Symmetric Multi-Processing)
- Advantage: very low communication cost; easy to program
- Limitation: memory bus becomes a bottleneck; does NOT scale beyond ~32–64 processors; expensive
- Example: High-end SMP servers, Oracle RAC on single machine

2. Shared-Disk Architecture

- Each processor has its own private memory but all processors share the same disks via a high-speed interconnect (SAN — Storage Area Network)
- Processors communicate by reading/writing shared disk; or via message passing
- Advantage: good fault tolerance — any processor can access any data; scales to more processors than shared-memory
- Limitation: disk interconnect can become bottleneck; cache coherency problem (each processor has private cache of disk pages — must be kept consistent)
- Example: Oracle RAC (Real Application Clusters), IBM DB2 pureScale

3. Shared-Nothing Architecture

- Each node (processor) has its own private memory AND its own private disks — no sharing
- Nodes communicate only via a high-speed message-passing network
- Data is partitioned across nodes — each node owns a subset of the database
- Advantage: scales to thousands of nodes (massively parallel); no shared resource bottleneck; high fault tolerance
- Limitation: queries that need data from multiple nodes require network communication (data shuffling); harder to implement and tune
- Example: Teradata, Google BigQuery, Amazon Redshift, Apache Spark, Hadoop

Feature	Shared Memory	Shared Disk	Shared Nothing
Memory	Shared global RAM	Private per processor	Private per node
Disk	Shared	Shared via SAN	Private per node
Communication	Shared RAM (fastest)	Disk + messages	Messages only
Scalability	Low (32–64 CPUs)	Medium (100s CPUs)	Very high (1000s nodes)
Cost	Very high (SMP hardware)	High (SAN hardware)	Lower (commodity hardware)
Fault tolerance	Low (single memory failure)	High (any CPU can access data)	Very high (node failures isolated)
Programming complexity	Low	Medium	High
Best for	OLTP, small-medium DBs	OLTP, high availability	Large-scale analytics, big data

Q: What is Object-Relational Mapping (ORM)? Explain how it works and its advantages. Give examples of popular ORM frameworks.
Asked: 2077 Chaitra Q9b; 2071 Magh Q10a — appears as short note | Marks: [3]

Answer

Object-Relational Mapping (ORM)

Object-Relational Mapping (ORM) is a programming technique that automatically maps between objects in an object-oriented programming language (Java, Python, C#) and rows in a relational database. ORM acts as a bridge (middleware layer) between the application layer (OOP) and the database layer (relational tables), eliminating most of the manual SQL code.

The Impedance Mismatch Problem

Object-oriented programming and relational databases represent data differently — this gap is called the object-relational impedance mismatch:

OOP Concept	Relational DB Concept	Mismatch Issue
Class	Table	Class has methods; table has none
Object (instance)	Row (tuple)	Objects have identity (OID); rows use key values
Attribute	Column	OOP allows nested objects; SQL requires atomic values
Inheritance (IS-A)	No direct equivalent	Must flatten to tables or use multiple tables
Association (HAS-A)	Foreign key + JOIN	Object references are direct; DB needs JOIN
Collection (List, Set)	Related table	Object holds collection; DB needs separate table

How ORM Works

- The developer defines classes (entities) with annotations or configuration files
- The ORM framework automatically generates SQL CREATE TABLE, INSERT, SELECT, UPDATE, DELETE statements
- The developer works entirely with objects — the ORM translates to/from SQL transparently

```
// Without ORM (manual SQL — tedious, error-prone):
String sql = "INSERT INTO employees (name,salary,dept_id) VALUES (?,?,?)";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, emp.getName());
ps.setDouble(2, emp.getSalary());
...

// With ORM (Hibernate / JPA — clean, simple):
@Entity
public class Employee {
  @Id int empId;
  String name;
  double salary;
  @ManyToOne Department dept;  // ORM handles the JOIN automatically
}
```

```
// Save: just persist the object — ORM generates INSERT
entityManager.persist(employee);

// Query: JPQL (object-based query)
List<Employee> emps = em.createQuery(
  "SELECT e FROM Employee e WHERE e.salary > 50000").getResultList();
```

Popular ORM Frameworks

Framework	Language	Database Support
Hibernate / JPA	Java	All major RDBMS
SQLAlchemy	Python	All major RDBMS
Entity Framework	C# / .NET	SQL Server, SQLite, PostgreSQL, MySQL
Django ORM	Python	PostgreSQL, MySQL, SQLite, Oracle
Active Record	Ruby (Ruby on Rails)	All major RDBMS
TypeORM	TypeScript / JavaScript	All major RDBMS + MongoDB

Advantages and Disadvantages

Advantages	Disadvantages
No manual SQL — faster development	Performance overhead vs hand-optimized SQL
Database-independent code (switch DB easily)	Complex queries still need native SQL
Prevents SQL injection attacks	Learning curve for ORM-specific query language
Automatic schema generation and migration	N+1 query problem if lazy loading not managed
Works naturally with OOP patterns	Heavy ORM (Hibernate) adds startup overhead

SUMMARY — All PDF Gaps Now Covered

This document covers all questions and topics found in the PDF question bank (2070–2082) that were NOT in the previous chapter answer banks:

#	Topic	Chapter	Asked In
G7.1	3-scenario log recovery: (a) no commit, (b) one commit, (c) both commit	Ch7/Ch8	2080, 2076, 2074, 2070 Bhadra
G7.2	Log recovery with checkpoint + T0/T1/T2 abort scenario	Ch8	2070 Bhadra, 2073 Magh
G7.3	Conflict serializability — schedule r1(y);r3(z);w1(y);w2(z);r3(y);w2(y)	Ch7	2079 Chaitra, 2072 Magh
G7.4	Precedence graph with 5 nodes — cycle detection	Ch7	2079 Jestha, 2077 Chaitra
G7.5	Graph-based (tree) protocol for concurrency control	Ch7	2079 Ashwin, 2077 Chaitra
G8.1	Remote backup system — architecture, hot-spare, warm-spare, RPO/RTO	Ch8	2079, 2077, 2075, 2073, 2070 — most exams
G8.2	Full worked log recovery problem (6-mark) with 3 transactions + checkpoint	Ch8	2070 Bhadra, 2073 Magh
G9.1	Homogeneous vs heterogeneous distributed databases; federated DB	Ch9	2079 Jestha, 2075 Baisakh, 2073 Magh, 2072 Magh, 2071 Magh
G9.2	Spatial database — data types, R-tree, spatial SQL, applications	Ch9	2075, 2073, 2072, 2071, 2070 — every older exam
G9.3	Parallel DB architectures — shared-memory, shared-disk, shared-nothing	Ch9	2079, 2077, 2072, 2071, 2070 — every older exam
G9.4	Object-Relational Mapping (ORM) — concept, frameworks, advantages	Ch9	2077 Chaitra, 2071 Magh

Quick exam tips: (1) 3-scenario log problem [6 marks] — always draws from 2080/2076/2070 log: scenario (a) UNDO T0, (b) REDO T0 + UNDO T1, (c) REDO both. (2) Remote backup system appears in nearly every exam as 3-mark short note — memorise: log shipping, hot/warm/cold spare, RPO/RTO. (3) Spatial DB and Parallel DB appear as [2–3 mark] short notes in older exams — know definitions + 3 architectures for parallel DB. (4) Conflict serializability: draw precedence graph → check for cycle → if no cycle, topological sort gives serial order.

— End of Supplementary Answer Bank —